

This is much like the `.exe` file for the Windows apps. All you need to do is to download the AppImage file, make it executable, and start running it.

Since AppImages are distro-agnostic, they can run in all available distros. They need not be installed in the system. Users can carry these with them and start working in a completely new system.

AppImages are bundled with all the pieces that are required to run that app. As they do not depend on the system libraries or files, they can run in an isolated environment without disturbing the host system. Any user can run these apps without the native permission restriction of the Linux system.

Also, AppImages are independent universal packaging systems; so users get to use the recent version of the app without having to wait for it to be included in the respective distro packaging.

Working with AppImages

To start working with AppImages, all you need to do is download an AppImage file and make it executable. This can be done via GUI or console. In the GUI way, you need to right-click on the AppImage and go to the 'Permissions' tab of the file, check the option 'Allow this file to run as a program', close the dialogue box, and double click on the AppImage to run it. The console way is fairly simple. Just issue the following command in your console:

```
$ chmod a+x AppImage_Name.AppImage
```

After the execution of the above command, the AppImage becomes executable, and to run it, issue the following command:

```
$ ./AppImage_Name.AppImage
```

When you double-click on them, some AppImages will ask you to install a desktop file. If you click on 'Yes', the respective AppImage will be added to your *Application* menu like an installed app.

Once you've decided to uninstall an AppImage, you can just delete the AppImage file from the system. However, if you've chosen desktop integration for that file, you may leave some residual files in the system, and the deletion of these files is entirely optional.

Each AppImage is executed by mounting the AppImage file with FUSE (File System in User Space), which enables the user to create a file system at the user level, without touching the host file system at the kernel level.

Sometimes a user may want to take a look under the hood of an AppImage. For that, the AppImage needs to be extracted. To extract its contents, we need to provide the path to the AppImage file and choose the 'extract' option from the AppImage *help* menu.

For example, we can take the Museeks AppImage, which is placed in a folder called 'Test_Table'.

To display the *help* menu of Museeks, the following command should be issued:

```
$ /home/tuxmachine/Test_Table/
museeks-x86_64.AppImage.AppImage
--appimage-help
```

To extract the AppImage, the user needs to issue the following command in the console:

```
$ /home/tuxmachine/Test_Table/
museeks-x86_64.AppImage --appimage-
extract
```

This will extract the contents of the AppImage and place them in a folder called *squashfs-root*.

However, the extracting option is not available for all the AppImages. So if it is not possible via console, you can use the *appimagetool* to extract the contents.

Portable mode

AppImage has another awesome feature called 'portable mode'. This enables the user to carry the app data along with the app itself. This feature is available in the recent versions of AppImage (built in 2017 or later).

Generally, the portable mode is enabled by creating a directory along with the extension (*.home* or *.config*) in the AppImage location. For example, if you want to enable the portable mode in the *textosaurus* app, then as a first step you need to change your current directory to the AppImage location and

```
AppImage options:
--appimage-extract [<pattern>] Extract content from embedded filesystem image
                                If pattern is passed, only extract matching files
--appimage-help                Print this help
--appimage-mount               Mount embedded filesystem image and print
                                mount point and wait for kill with Ctrl-C
--appimage-offset              Print byte offset to start of embedded
                                filesystem image
--appimage-portable-home       Create a portable home folder to use as $HOME
--appimage-portable-config     Create a portable config folder to use as
                                $XDG_CONFIG_HOME
--appimage-signature           Print digital signature embedded in AppImage
--appimage-updateinfo[rmation] Print update info embedded in AppImage
--appimage-version             Print version of AppImageKit

Portable home:

If you would like the application contained inside this AppImage to store its
data alongside this AppImage rather than in your home directory, then you can
place a directory named

/home/arulmagi/Test_Table/museeks-x86_64.AppImage.home

Or you can invoke this AppImage with the --appimage-portable-home option,
which will create this directory for you. As long as the directory exists
and is neither moved nor renamed, the application contained inside this
AppImage to store its data in this directory rather than in your home
directory
```

Figure 1: AppImage *help* menu